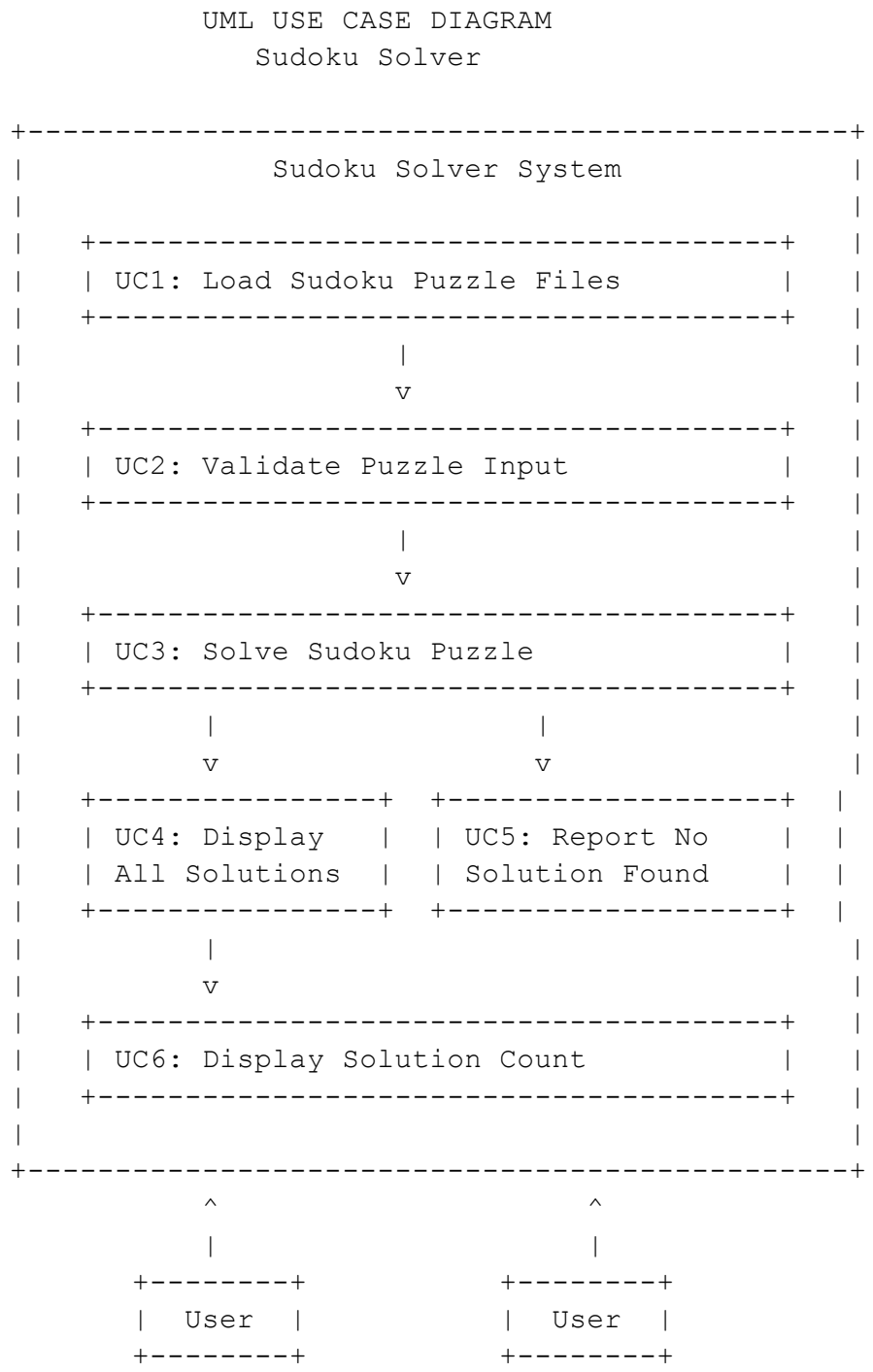
UML Use Case Diagram:



Use case name: Solve Sudoku Puzzles

Primary actor: User

Goal: Find and display all valid solutions to Sudoku puzzles stored in puzzle1.txt through puzzle5.txt.

Trigger: The user runs the Sudoku Solver program.

Preconditions:

The program has been compiled successfully.

The puzzle files are available in the expected directory.

Each puzzle file contains 81 cell values, with underscores representing empty cells.

Postconditions:

The original puzzle is displayed.

Every valid solution is displayed, or a no-solution message is shown.

The total number of solutions is reported.

Use Case Description:
The user runs the Sudoku Solver program, which reads each puzzle file, validates the input and pre-filled digits, and attempts to solve the puzzle using recursive backtracking. The system prints all valid solutions and the total solution count. If the puzzle is invalid or has no solution, the system reports the appropriate error or no-solution result.

Step-by-step flow of events:

1. The user starts the program.
2. The system identifies the five puzzle filenames.
3. The system opens the first puzzle file.
4. The system reads the 81 cell values.
5. The system checks the input format and validates the pre-filled digits.
6. The system prints the original puzzle.
7. The solver identifies an empty cell with the fewest valid candidates.
8. The solver tries a candidate and recursively searches for a solution.
9. If a candidate leads to a dead end, the solver backtracks and tries another candidate.
10. When a complete valid board is found, the system prints the solution and increments the solution count.
11. The system continues searching until all candidate paths have been explored.

12. The system prints the total number of solutions, or No solution found if no solution exists.

13. The system repeats the process for the remaining puzzle files.

14. The program terminates after processing all five files.

Detailed description:

Use case ID:

UC-01

Use case name:

Solve Sudoku Puzzles

Primary actor:

User

Supporting system:

File system

Description:

Reads, validates, solves, and displays Sudoku puzzles.

Trigger:

The user executes the program.

Preconditions:

The program is compiled and the puzzle files are accessible.

Main flow:

Load puzzle, validate input, display original board, recursively solve, print all solutions, and report the solution count.

Alternative flow A:

If a puzzle file cannot be opened, the system reports an error and continues with the next file.

Alternative flow B:

If the puzzle format is invalid or the initial clues conflict, the system reports an error and skips that puzzle.

Alternative flow C:

If a valid puzzle has no solution, the system displays No solution found and reports zero solutions.

Postconditions:

The results for each successfully loaded and validated puzzle are displayed.

Main success scenario:
The user starts the program. The system reads and validates a Sudoku puzzle, prints the original board, and uses recursive backtracking to explore candidate values. Whenever a complete valid solution is found, it is displayed. The solver continues exploring until all possible solutions have been found. Finally, the system displays the number of solutions and proceeds to the next puzzle file.

Alternative scenarios:
File unavailable: The system prints an error message and moves on to the next file.

Invalid input: The system rejects the malformed puzzle.

Conflicting initial clues: The system rejects the puzzle before starting the recursive search.

No valid solution: The system reports that no solution was found.

Multiple solutions: The system displays every solution and reports the total count.